



Research Report - Semester 2

Adaptive Workplace Dynamics Simulator

Exploratory agent-based simulation of organizational dynamics

Master student:

Pop Andrei

Supervisor:

Anca Andreea Morar

June 18, 2026

Abstract

This document describes the Adaptive Workplace Dynamics Simulator (AWDS) prototype, a Python application with a Streamlit interface for synthetic simulation of workplace dynamics. The report presents the implemented features, the model, the technical structure, encountered difficulties, limitations, and future development directions. Numerical results are intended to be completed after the final scenarios are run.

Contents

1	Prototype Description	2
2	Research Context and Objective	2
3	Implemented Features	2
4	Simulation Model	3
4.1	Dynamic Variables	3
4.2	Static Factors	4
4.3	Model Assumptions	4
4.4	General Logic	4
4.5	Turnover	4
4.6	Reproducibility	5
5	Technical Implementation	5
5.1	Project Structure	5
5.2	Simulation Engine Architecture	5
5.3	Mesa Alternative	6
5.4	Shared Output Schema	6
5.5	Dashboard and Export Workflow	6
6	Scenarios and Results	7
6.1	Scenario Design	7
6.2	Metric Interpretation	7
6.3	Planned Experiment Workflow	8
6.4	Experimental Configuration	9
6.5	Comparison Table	9
6.6	Graphs	9
6.7	Interpretation	9
7	Encountered Difficulties	9
8	Limitations	10
9	Future Directions	10
10	Video Demo	11

1 Prototype Description

AWDS is a software prototype that simulates an organization made of employee agents. Each agent has an internal dynamic state, including stress, burnout, productivity, emotional state, energy, job satisfaction, work-life balance, and engagement.

The application is built as an exploratory tool. Its purpose is not to prove real organizational relationships, but to observe synthetic dynamics generated by a configurable model. Therefore, the results should be interpreted as model outputs, not empirical data.

The interface allows the user to configure parameters, run simulations, view graphs, compare scenarios, and export results for later use in a report.

2 Research Context and Objective

Workplace dynamics are difficult to study because many relevant variables influence each other at the same time. Workload, deadline pressure, management support, autonomy, recovery, family responsibilities, and social relations do not act as isolated causes. They form a system in which short-term productivity can improve under pressure, while the same pressure can create burnout risk when it persists for longer periods.

The prototype is designed around this systems perspective. Instead of estimating a single final value, AWDS follows a population of synthetic employee agents across simulated workdays. This makes it possible to observe trajectories: how stress changes over time, how burnout accumulates, how productivity reacts to pressure, and how turnover emerges after repeated exposure to unfavorable conditions.

The research objective for this stage is to build a transparent simulation platform that can support controlled comparisons between organizational scenarios. The focus is not prediction, but exploration. A useful prototype should make it easy to ask questions such as: what changes when management support increases, what happens when overtime and deadline pressure are combined, and whether the same scenario tendency appears when the model is executed through a different simulation engine.

For this reason, the current implementation emphasizes four requirements:

- clear configuration of organizational, personal, social, and event-related parameters;
- reproducible runs based on explicit seeds and saved presets;
- comparable outputs across scenarios and engines;
- exportable data and figures that can be used in the dissertation report.

3 Implemented Features

The prototype includes the following main features:

- configuration for number of agents, number of simulated days, and random seed;
- factory presets for predefined organizational scenarios;

- local user presets saved in `user_presets.json`;
- explicit preset application button to avoid accidental parameter changes;
- saving the current configuration as a new preset;
- sticky controls for run, pause/resume, and clearing the displayed run;
- visible slider range editing through the *Advanced slider ranges* panel;
- live graphs for stress, burnout, productivity, and secondary indicators;
- engine selector for the custom engine, the Mesa engine, or both engines in sequence;
- scenario comparison using the same seed and a selected comparison engine;
- direct comparison between engine outputs when both engines are selected;
- completed run history on the export page;
- search in export history by scenario, timestamp, seed, day, number of agents, or ID;
- export to JSON, CSV, TXT, PNG, and ZIP;
- startup scripts for Unix/macOS and Windows: `run.sh` and `run.bat`.

4 Simulation Model

Each simulation step represents one simulated workday. On each day, the model applies organizational factors, personal factors, social influence, random events, and update rules for each agent.

4.1 Dynamic Variables

Agents have the following main dynamic variables:

- **stress**: stress level, normalized between 0 and 1;
- **burnout**: burnout accumulation, normalized between 0 and 1;
- **productivity**: current productivity, normalized between 0 and 1;
- **emotion**: emotional state, between -1 and 1;
- **energy**: energy or recovery capacity, between 0 and 1;
- **satisfaction**: job satisfaction, between 0 and 1;
- **work_life_balance**: work-life balance, between 0 and 1;
- **engagement**: employee engagement, between 0 and 1.

4.2 Static Factors

Each agent is initialized with different personal traits: role, resilience, ambition, need for social support, commute sensitivity, family load, health variability, baseline productivity, stress sensitivity, and recovery rate.

4.3 Model Assumptions

The model intentionally uses simplified assumptions so that the internal behavior remains inspectable. Each agent is synthetic, and its traits are generated from simple statistical distributions rather than from real personal records. This keeps the prototype appropriate for experimentation before empirical calibration is available.

The main assumptions are:

- most psychological and organizational quantities are normalized between 0 and 1, while emotion ranges from -1 to 1;
- the social network is static during a run, so the model does not yet simulate new friendships, conflicts, or team restructuring;
- random events are probabilistic shocks that temporarily affect stress, emotion, energy, resources, or demands;
- turnover is a model event triggered by stress and burnout risk, not a prediction of real employee behavior;
- management style and work policy are parameter modifiers, not separate institutional agents.

These assumptions limit realism, but they also make the prototype easier to test. At this stage, transparency is more important than complexity. A more complex model would only be useful after the simpler version is understood and after there is empirical data that can justify additional parameters.

4.4 General Logic

The model uses simple and transparent equations. Stress increases based on work demands, external pressure, and negative events, but decreases based on resources, recovery, and resilience. Burnout accumulates more slowly than stress and is influenced by sustained high stress. Productivity is not modeled as a linear “stress is always bad” relationship. The model includes an optimal pressure effect: moderate stress may support productivity in the short term, while high stress and burnout reduce productivity.

4.5 Turnover

In the application, *turnover* means the cumulative number of simulated employees who leave the organization. An agent may leave when burnout or stress exceeds configured thresholds. If replacement is enabled, a new agent joins after a configured delay and temporarily receives an onboarding productivity penalty.

4.6 Reproducibility

The custom engine uses `numpy.random.default_rng(seed)`, while the Mesa engine receives the same configured seed through the Mesa model. The same seed, configuration, and selected engine produce the same result series for that engine. The same seed is not expected to produce identical trajectories across the custom and Mesa engines, because the implementation and activation order are different. This property is useful for comparing scenarios under controlled conditions while also comparing how stable the observed tendencies are across engine implementations.

5 Technical Implementation

The application is implemented in Python. The web interface is built with Streamlit, and numerical calculations are handled with NumPy. Data is organized and exported with Pandas, while exportable charts are generated with Matplotlib. Plotly is used for interactive dashboard charts when available. The Mesa implementation uses Mesa model and agent classes together with NetworkX-backed network space.

5.1 Project Structure

```
app.py
run.sh
run.bat
simulation/
  config.py
  agent.py
  engine.py
  mesa_engine.py
  engines.py
  scenarios.py
  metrics.py
  exports.py
  utils.py
exports/
requirements.txt
```

5.2 Simulation Engine Architecture

The prototype now separates the simulation logic behind an engine interface. An engine exposes the same basic operations: advancing one day, running until completion, returning a live snapshot, and returning the final result. The dashboard uses `engines.py` to map the selected engine name to the implementation that should run. This makes the interface independent from the exact engine implementation.

The custom engine, implemented in `engine.py`, is a lightweight local implementation. It directly manages the list of employee agents, the social network, random events, daily state updates, turnover, replacement, and aggregate metrics. Its main advantage is transparency: the update equations and output format are easy to inspect and tune.

5.3 Mesa Alternative

Mesa has also been added as an alternative engine in `mesa_engine.py`. In this version, the workplace is represented as a Mesa model, employees are Mesa agents, and relationships are stored in a network grid backed by NetworkX. The Mesa engine preserves the same result fields as the custom engine, so the dashboard, exports, and comparison tables can treat both engines uniformly.

The goal of adding Mesa is not to make the outputs identical. Instead, the goal is to compare scenario tendencies across two implementations. If the custom and Mesa engines show similar directions under the same preset, seed, and run controls, the model behavior is easier to interpret. If the outputs differ, that difference helps identify which assumptions or implementation details need closer research.

5.4 Shared Output Schema

Both engines return the same categories of output: the project name, engine name, scenario name, seed, configuration, aggregate time series, optional per-agent history, final agent states, event log, and final summary. This shared schema is important because it prevents the dashboard from becoming two separate applications. The same chart code, export code, run history, and comparison tables can be reused regardless of the selected engine.

The final summary contains the metrics that are most useful for scenario comparison: final average stress, final average burnout, final average productivity, total turnover, cumulative burnout, and lowest productivity. The aggregate time series is kept separately so that the report can include both final values and trajectories.

5.5 Dashboard and Export Workflow

The Streamlit dashboard is organized around the normal workflow of an exploratory simulation session. The user first chooses a preset or manually adjusts the parameters in the sidebar. Presets are applied explicitly, which avoids changing the current configuration by accident while browsing through possible scenarios. The same sidebar also controls the selected engine, the number of agents, the number of simulated days, the seed, and the main behavioral switches such as emotional contagion, turnover, replacements, and random events.

After a run is completed, the dashboard presents both high-level summary metrics and time-series charts. The most important chart overlays stress, burnout, and productivity so that the user can see whether pressure produces short-term productivity gains, longer-term burnout, or both. Supporting charts show emotional state, satisfaction, work-life balance, energy, active employee count, and turnover. This structure keeps the interface focused on interpretation rather than only on raw numbers.

The export tab is part of the research workflow, not only an extra feature. Each completed run is stored in a session history with a readable timestamp, scenario name, engine name, seed, and run ID. The user can search the history and prepare an export bundle for a selected run. The bundle includes machine-readable data, a text summary, and chart images, which makes the output suitable for later inclusion in the report.

This workflow matters because the prototype is expected to support iteration. The user can run a scenario, inspect the trajectory, adjust parameters, run an alternative en-

gine, and then export the results that are most relevant for discussion. The dashboard therefore acts as both a simulation interface and a lightweight research notebook.

6 Scenarios and Results

This section is a skeleton to be completed after running the final scenarios and exporting the graphs.

The comparison workflow is central to the prototype. Scenarios can be compared with the custom engine, with the Mesa engine, or with both engines. This gives a clearer view of whether the research focus should be on a scenario effect itself or on differences introduced by the simulation engine.

6.1 Scenario Design

The factory scenarios are not meant to represent measured organizations. They are controlled configurations designed to stress different parts of the model. A balanced baseline gives a neutral reference point, supportive scenarios test the protective effect of resources, and high-pressure scenarios test the accumulation of stress and burnout under unfavorable conditions.

Scenario type	Purpose in the prototype
Balanced baseline	Provides a neutral reference for comparing changes in management style, workload, and work policy.
Supportive management	Tests whether higher autonomy, recognition, role clarity, and psychological safety reduce stress and burnout.
Authoritarian or overtime-heavy management	Tests the effect of high demands, reduced autonomy, deadline pressure, and weaker support.
Remote or flexible work	Explores how commute impact, flexibility, and meeting load affect well-being and productivity.
Toxic workplace	Pushes conflict, low safety, low support, and high pressure to observe whether the model produces visible deterioration.

Table 1: Scenario design logic used by the AWDS presets.

The scenario comparison tab keeps the number of agents, number of days, seed, and selected run controls constant while changing the preset-specific organizational parameters. This isolates the effect of the scenario as much as possible inside the synthetic model. The separate comparison engine selector makes it possible to run the same comparison through the custom engine, the Mesa engine, or both.

6.2 Metric Interpretation

The exported metrics should be interpreted as internal model indicators. They are useful because they summarize different parts of the simulated workplace dynamics:

Metric	Interpretation
Final average stress	Average stress among active agents at the end of the run. Useful for comparing final pressure levels.
Final average burnout	Average accumulated burnout at the end of the run. Useful for longer-term deterioration.
Final average productivity	Average productivity at the end of the run. Should be read together with stress and burnout.
Total turnover	Number of simulated employees who left during the run. Indicates whether stress and burnout thresholds were crossed often enough to trigger exits.
Cumulative burnout	Area-like accumulation of burnout over time. Useful when two scenarios end similarly but one had worse conditions during the run.
Lowest productivity	Minimum aggregate productivity observed during the run. Useful for identifying short-term collapse or disruption.

Table 2: Interpretation guide for the main AWDS comparison metrics.

6.3 Planned Experiment Workflow

A complete experiment should start from a small number of stable presets rather than from many manually changed parameters. First, the balanced baseline should be run with a fixed seed and a fixed number of agents and days. This creates a reference trajectory for stress, burnout, productivity, and turnover. Then each scenario should be run with the same seed and run controls so that differences are easier to attribute to the scenario configuration.

The next step is to repeat the comparison with both engines. The purpose is not to force identical values, but to observe whether the direction of the result remains similar. For example, if the toxic workplace scenario produces higher burnout than the balanced baseline in both engines, that tendency is more useful for discussion. If one engine produces a much stronger or weaker effect, the difference should be documented and used to inspect the assumptions that produce it.

The proposed workflow is:

1. choose a fixed number of agents, number of days, and seed;
2. run the baseline scenario with the custom engine;
3. run the selected comparison scenarios with the same controls;
4. repeat the same scenario set with the Mesa engine;
5. export the comparison table and time-series charts;
6. interpret only model tendencies, not real-world causal claims.

This workflow keeps the report aligned with the current maturity of the prototype. It provides enough structure to discuss results while still acknowledging that calibration and validation remain future work.

6.4 Experimental Configuration

- Number of agents: *[to be completed]*
- Number of simulated days: *[to be completed]*
- Seed used: *[to be completed]*
- Engine or engines compared: *[to be completed]*
- Compared scenarios: *[to be completed]*
- Parameters changed from presets: *[to be completed]*

6.5 Comparison Table

Scenario	Engine	Final stress	Final burnout	Final productivity	Turnover	Cumulative burnout
<i>[Scenario 1]</i>	<i>[Engine]</i>	–	–	–	–	–
<i>[Scenario 2]</i>	<i>[Engine]</i>	–	–	–	–	–
<i>[Scenario 3]</i>	<i>[Engine]</i>	–	–	–	–	–

Table 3: Synthetic comparative results obtained from AWDS simulations.

6.6 Graphs

Insert exported application graphs here.

Placeholder for the stress–burnout–productivity graph.

Figure 1: Evolution of stress, burnout, and productivity for scenario *[to be completed]*.

6.7 Interpretation

[To be completed after running the final scenarios. The interpretation should describe synthetic model dynamics, compare custom and Mesa tendencies when both engines are used, and avoid presenting the outputs as empirical findings.]

7 Encountered Difficulties

The main difficulty was designing a model that was expressive enough not to reduce workplace dynamics to trivial rules such as “stress is bad” and “free time is good”. For the simulation to be useful, the model had to allow more nuanced situations: for example, moderate pressure may increase productivity in the short term, while high and sustained pressure may lead to burnout and lower productivity.

Another difficulty was balancing the parameters so scenarios produce visible differences without all runs immediately reaching extremes. It was necessary to separate

stress, burnout, energy, satisfaction, and productivity, because these variables do not all move at the same speed and do not respond identically to the same factors.

Interface configurability was also a practical challenge, because the number of parameters is large. Streamlit significantly simplified the UI side by enabling fast construction of configuration panels, charts, and controls. Still, the controls needed clear grouping, explicit presets, parameter explanations, and an export system that is easy to use.

Exporting results was treated as part of the user experience: the user should be able to return to runs, search them, identify each run by ID and timestamp, and download data in formats suitable for the report.

Adding a second engine introduced an additional implementation challenge: both engines had to produce the same output schema so that charts, exports, run history, and comparison tables could reuse the same code. This was important because the research value comes from comparing results, not from maintaining separate workflows for each implementation.

8 Limitations

The most important limitation is that the model uses synthetic data. Parameters and equations are configurable and reproducible, but they are not calibrated on real empirical data. For this reason, results can be used to discuss the internal behavior of the model, not to draw conclusions about real companies.

The model simplifies many aspects of organizational environments: social relationships are represented through a static network, events are modeled probabilistically, and agent traits are generated randomly from simple distributions. These decisions are appropriate for a prototype, but they limit simulation realism.

9 Future Directions

The most important next step would be obtaining empirical data for model calibration. One possible direction is creating a questionnaire distributed to people working in different companies, collecting information about perceived stress, workload, managerial support, autonomy, work-life balance, satisfaction, and intention to leave the organization.

The goal of such a questionnaire would be to turn subjective experiences into approximate values, in other words to “put a number on a feeling”. These values could be used to adjust model parameters, compare synthetic scenarios with real responses, and identify variables that should be represented more accurately.

Other development directions include:

- calibrating parameters based on collected responses;
- extending the model with teams, departments, and more detailed organizational roles;
- sensitivity analysis to identify which parameters influence results the most;
- systematic comparison between custom-engine and Mesa-engine outputs;

- persistent storage of runs in a database;
- improved visualizations and automatically generated reports;
- partial validation by comparing synthetic tendencies with anonymized real data.

10 Video Demo

The video demo will present the main application flow:

1. starting the application;
2. applying a factory preset;
3. manually adjusting parameters;
4. saving a local preset;
5. selecting the custom engine, Mesa engine, or both engines;
6. running a simulation;
7. pausing/resuming a live run;
8. comparing multiple scenarios with the selected comparison engine;
9. exporting results from an export drawer;
10. downloading JSON, CSV, TXT, PNG, and ZIP files.

Video demo link: *[to be completed]*